

P?S/8 System Programs Directory Internal Structures

Last edit: 04-Jun-2018 cjl

Unlike most PDP-8 operating systems, P?S/8 maintains a separate directory for core-image system programs. This allows highly specialized entries tailored to the individual requirements of each program. This includes certain command-line parsing variations and the ability to maintain internal program overlays. The P?S/8 command-line mechanism supports many features that are akin to Concise Command Language [CCL] as implemented in other systems with system-specific variations. Certain aspects of the commands are managed by control bits in the entries for each system program; this is especially crucial regarding programs implemented as overlays to the keyboard monitor.

Figure 1 shows the entire System Programs directory particular to the TC01/TC08 configuration and includes all specific program details. Note: This listing can be obtained using the P?S/8 *DIRECT* program with the /S command-line option switch (and possibly additional options) set; for more information, consult separate documentation of P?S/8 *DIRECT*.

P?S/8 System Directory DIRECT V10b Thursday 02-Jul-15 Page 1											
System: TC08:0											
Image Information:											
Name	Block	Size	Load	Length	Out	In	R	S	O	Start	
ODT	0163	016	0000	16	GENO	GENI	0	0	0	03000	
BATCH	0201	002	7400	01		BAT	0	0	0	07400	
BIN	0203	027	0000	13	BIN	BIN	0	0	0	00200	
GET	0203	027	0000	13	BIN	BIN	0	0	0	00600	
START	0203	027	0000	13	GENO	GENI	0	0	0	00400	
DUMP	0232	007	0000	07		DUMP	0	0	0	01400	
CHANGE	0241	003	1200	03			1	1	0	01600	
FIND	0241	003	1200	03			1	1	0	01200	
BSAVE	0244	006	3000	06	BIN		0	0	1	03000	
PAL	0252	102	0000	37	BIN	PAL	0	0	0	00200	
MAP	0354	012	0000	12		BIN	0	0	0	01200	
BLKODT	0366	011	0000	11			0	0	0	02000	
SET	0377	013	3000	13			1	1	0	03000	
CORE	0412	002	3000	02			0	1	0	03000	
DATE	0414	012	3000	12			0	1	0	03000	
PRINT	0426	014	0000	14		PAL	1	0	0	00200	
FILMAN	0442	024	2600	24			0	1	0	07000	
ALLCAT	0466	003	3000	03			0	1	0	03000	
CONSOL	0471	040	3000	20		BIN	0	0	0	03000	
FOCAL	0531	041	0000	37		FOC	0	0	0	03400	
OS8CON	0572	012	0000	12	ASC	ASC	0	0	0	00200	
SYSTAT	0604	014	0000	14			0	0	0	00200	
DIRECT	0620	023	2600	23			1	1	0	07200	
BLKCPY	0643	026	0000	26			0	0	0	00200	
RK4MAT	0671	021	0000	21			0	0	0	02200	
DT4MAT	0712	014	0000	14			0	0	0	02600	
DTCOPY	0726	007	0000	07			0	0	0	00000	
<Free>	0735	043									
Free Slots:		072									

Figure 1: System Directory Printout of P?S/8 Version 8Z for TC08.

Figure 2 shows the contents of the first block of the System Programs directory. Each program entry occupies six words:

- Word 1-3: The program name in six-bit text format. All trailing characters of program names shorter than six characters are nul [00]. Any character that is valid for a *Tiny File System* [TFS] file name is also valid for a system program name. This includes characters such as \, ^, [and].
- Word 4: The starting block number for the program. The contents of this block (and any relevant additional contiguous blocks) will be loaded into memory when an execution-class keyboard monitor command initiates the program.

Note: Since system programs in the basic P?S/8 system are loaded by calls to the system device handler, the starting block number is a 12-bit unsigned value. When the optional P?S/8 *SHELL* overlay is implemented, programs will be controlled by independent entries in the *SHELL* filesystem directory. As such, the starting block will be expressed as a 24-bit unsigned integer which will be passed to a designated non-system device handler.

Note: The additional words in each entry are likely to change in the next release; this may include changing the exact count of words per entry from six to seven. Due to the generous allocation of blocks in the system programs directory, this is not expected to be of any practical consequence.

Word 5: This word contains several sets of loading and parsing information:

Bits[0-4] contain the loading address of the system program (which must conform to full page boundaries).

Bits[7-11] contain the length (in 128-word blocks) of the initial loading segment of the program.

Note: All programs that do not utilize overlay structures contain no additional segments. Programs that include overlay mechanisms define their own internal structures as required. For example, P?S/8 *FOCAL* includes several secondary segments that are loaded into memory by FOCAL in specific circumstances; P?S/8 *PAL* has an analogous set of secondary segments of program code and a data segment that consists of the internal symbol table that is loaded into memory as determined by dynamic memory allocation considerations.

If Bit[5] is set, the command processor will not scan for (input or output) filenames. Certain programs are designed to process the rest of the command line directly. For example, the P?S/8 *CORE* program sets this bit to allow processing of various in-line arguments that must not be confused with filenames.

If Bit[6] is set, processing of an *=xxxx* expression will occur in the current radix setting within the keyboard monitor. Otherwise, the expression is always processed as octal notation. For example, the P?S/8 *FIND* program locates strings within file lines (within the edit buffer of the keyboard monitor) matching a specified string. The search can be limited to a starting line number expressed as *=xxxx* in the current radix setting within the keyboard monitor.

Word 6: This word contains several sets of startup and parsing parameters:

Bits[0-4] contains the startup address for the program. Note: As of P?S/8 Version 8Z, all system programs must start on full page boundaries. Future releases may relax this restriction somewhat.

If Bit[5] is set, the parsing of input files is inhibited. For example, the P?S/8 *BSAVE* program must be passed output files without input files. After the input/output separator character (< or > according to user preference) has been processed, the list of output files is always processed first. All command-line input beyond the separator character must be available to the BSAVE program for direct processing. This section of the command consists of individual memory locations or groups of contiguous memory locations to be saved in the specified binary output files; however, if input file processing were not inhibited, certain memory location specifications could be mistaken for input files.

Bits[6-11] are used for passive cosmetic purposes only. Various combinations of potential input and output file types have been defined. The P?S/8 *DIRECT* program output as shown in Figure 1 includes the designated values for all programs that are ordinarily used with input files or output files (or both).

Future P?S/8 releases will drop support of the usage of Bits[6-11] in word 6 as described above. These bits will be repurposed to allow the program starting address to be any value within field 0. This will be implemented by moving the special scan bits to a seventh word; this will also allow for future system expansion such as theoretical SHELL compatibility allowing the SHELL subsystem to access compatible programs from the System Programs directory.

Note: SHELL-based executable files will support a starting address anywhere within the entire 32K PDP-8 memory space.

```

BLOCK: 0:0063
0000 1704 2400 0000 0163 0016 3001 0201 2403 *ODT@@@A3@NXABATC*
0010 1000 0201 7401 7411 0211 1600 0000 0203 *H@BA<A<IBIN@@@BC*
0020 0013 0203 0705 2400 0000 0203 0013 0603 *@KBCGET@@@BC@KFC*
0030 2324 0122 2400 0203 0013 0401 0425 1520 *START@BC@KDADUMP*
0040 0000 0232 0007 1412 0310 0116 0705 0241 *@@BZ@GLJCHANGE!*
0050 1343 1600 0611 1604 0000 0241 1343 1200 *K#N@FIND@B!K#J@*
0060 0223 0126 0500 0244 3006 3104 2001 1400 *BSAVE@B$XFYDPAL@*
0070 0000 0252 0037 0207 1501 2000 0000 0354 *@@B*_@_BGMAP@@@C,*
0100 0012 1202 0214 1317 0424 0366 0011 2000 *@JJBBLKODTC6@IP@*
0110 2305 2400 0000 0377 3153 3000 0317 2205 *SET@@@C?Y+X@CORE*
0120 0000 0412 3102 3000 0401 2405 0000 0414 *@@DJYBX@DATE@@DL*
0130 3112 3000 2022 1116 2400 0426 0054 0215 *YJX@PRINT@DV@,BM*
0140 0611 1415 0116 0442 2724 7000 0114 1403 *FILMAND"WT8@ALLC*
0150 0124 0466 3103 3000 0317 1623 1714 0471 *ATD6YCX@CONSOLD9*
0160 3020 3002 0617 0301 1400 0531 0037 3413 *XPXBFOCAL@EY_\K*
0170 1723 7003 1716 0572 0012 0205 2331 2324 *OS8CONE:@JBESYST*

```

Figure 2: Dump of Block 0063 of P?S/8 Version 8Z for TC08.

Figure 3 shows the contents of the second block of the System Programs directory. Several empty entries exist in this block indicating there is ample room for additional system programs.

Note: To add a significant amount of additional system programs, the system device must be much larger than a DECTape or other relatively small storage media. Larger devices can address up to block 7777 which would allow significant additional image program support.

P?S/8 execution is initiated by one of two methods: concise command execution (starting the command with the program name), or by use of the R (or RU{N}) command specifying the desired program by name.

Note: In the current release, the RU{N} command is an alias for the R command; either can be used interchangeably.

When the R (or RU{n}) command method is used, the entire directory is available for system programs; in this mode of usage, a total of 125 octal (85 decimal) system programs can be accommodated. Figure 1 indicates 072 octal (58 decimal) slots are available as the System Programs directory currently contains 33 octal (27 decimal) programs.

When concise command execution is used, a restriction is made to only access entries wholly contained in the first two blocks of the System Programs directory. (Note: Any entry past relative word[173] of the second System Programs directory block is destroyed in memory by the command processor.) As such, when concise command execution is used, the maximum number of accessible slots is reduced to 52 octal (42 decimal). With regard to the System Programs directory used in the example, 17 octal [15 decimal] programs can still be added while also avoiding the possibility of being unavailable for the concise execution method.

Note: The two methods of initiating program execution have several tradeoffs:

- 1) Certain programs are implemented as overlays to the keyboard monitor such as the *FIND* program used to locate string matches within the tentative file edit buffer of the keyboard monitor. Only the concise execution method preserves the contents of the edit buffer; attempting to use *FIND* via the method of R *FIND* would be moot as the edit buffer contents will be destroyed.

- 2) Certain programs such as P?S?8 *OS8CON* can be executed to make use of the extended user directory for the purpose of creating variable-length files. The concise execution form cannot access the extended-length file directory.

The general recommendation is to place programs with little interaction with P?S/8 resources in higher blocks to ensure programs implemented as overlays to always be executable by the concise method. These generally include device formatting or diagnostic programs designed for standalone usage (with the paper-tape loading programs *RIM* or *BIN*.)

Note: The support of extended-length files is undergoing major redesign. As originally intended, it was foreseen that P?S/8 might attempt to include certain aspects of variable-length files in a more general-purpose way than merely one extended-length file per system device logical unit; however, no version of P?S/8 ever supported more than read access to the first file and the file name had to be manually entered into the first entry in the extended-length files directory.

With the implementation of the P?S/8 SHELL (currently under development) the extended-length files directory is being eliminated in favor of a single entry placed at the end of the System Programs directory. As such, the maximum number of system programs available to the R (or RU{n}) command will be reduced from 125 octal (85 decimal) to 123 octal (83 decimal). Since the entry for the single extended-length file will be loaded into memory along with the rest of the System Programs directory, overall command efficiency will be notably improved.

Appropriate modifications will be made to the P?S/8 *SET* command to accommodate the single extended-length file; it will no longer be necessary to enter the file name manually.

Due to this design change, the R (or RU{n}) form of execution cannot access the extended-length directory presently. A workaround is available as follows:

The TFS directory manages all blocks up to the limit block just before the extended-length file directory area. As such, a TFS file entry can be placed at the last allowed file slot. This file entry will start exactly 20 octal (16 decimal) blocks before the (first and only) extended-length file. As such, this final TFS directory slot can be used in lieu of an extended-length file name as necessary.

Note: The file will not be compatible with the keyboard monitor edit-oriented commands. All system programs can be passed this file in the same manner an extended-length file would ordinarily be used.

Note: P?S/8 OS8CON and related programs have been altered to not check validity of the passed file block; when access to the extended-length single file entry is restored, OS8CON validity checking will be restored.

```
BLOCK: 0:0064
0000 0124 0604 0014 0200 0411 2205 0324 0620 *ATFD@LB@DIRECTFP*
0010 2763 7200 0214 1303 2031 0643 0026 0200 *W3:0BLKCPYF#@VB@*
0020 2213 6415 0124 0671 0021 2200 0424 6415 *RK4MATF9@QR@DT4M*
0030 0124 0712 0014 2600 0424 0317 2031 0726 *ATGJ@LV@DTCOPYGV*
0040 0007 0000 0000 0000 0000 0000 0000 0000 *@C@C@C@C@C@C@C@C@C@*
0050 0000 0000 0000 0000 0000 0000 0000 0000 *@C@C@C@C@C@C@C@C@C@*
0060 0000 0000 0000 0000 0000 0000 0000 0000 *@C@C@C@C@C@C@C@C@C@*
0070 0000 0000 0000 0000 0000 0000 0000 0000 *@C@C@C@C@C@C@C@C@C@*
0100 0000 0000 0000 0000 0000 0000 0000 0000 *@C@C@C@C@C@C@C@C@C@*
0110 0000 0000 0000 0000 0000 0000 0000 0000 *@C@C@C@C@C@C@C@C@C@*
0120 0000 0000 0000 0000 0000 0000 0000 0000 *@C@C@C@C@C@C@C@C@C@*
0130 0000 0000 0000 0000 0000 0000 0000 0000 *@C@C@C@C@C@C@C@C@C@*
0140 0000 0000 0000 0000 0000 0000 0000 0000 *@C@C@C@C@C@C@C@C@C@*
0150 0000 0000 0000 0000 0000 0000 0000 0000 *@C@C@C@C@C@C@C@C@C@*
0160 0000 0000 0000 0000 0000 0000 0000 0000 *@C@C@C@C@C@C@C@C@C@*
0170 0000 0000 0000 0000 0000 0000 0000 0000 *@C@C@C@C@C@C@C@C@C@*
```

Figure 3: Dump of Block 0064 of P?S/8 Version 8Z for TC08.

Figure 4 shows the contents of the third system programs directory block which is provided for completeness. No additional entries exist within the example directory past the second system programs directory block.

```
BLOCK: 0:0065
0000 0000 0000 0000 0000 0000 0000 0000 0000 *@C@C@C@C@C@C@C@C@C@*
0010 0000 0000 0000 0000 0000 0000 0000 0000 *@C@C@C@C@C@C@C@C@C@*
0020 0000 0000 0000 0000 0000 0000 0000 0000 *@C@C@C@C@C@C@C@C@C@*
0030 0000 0000 0000 0000 0000 0000 0000 0000 *@C@C@C@C@C@C@C@C@C@*
0040 0000 0000 0000 0000 0000 0000 0000 0000 *@C@C@C@C@C@C@C@C@C@*
0050 0000 0000 0000 0000 0000 0000 0000 0000 *@C@C@C@C@C@C@C@C@C@*
0060 0000 0000 0000 0000 0000 0000 0000 0000 *@C@C@C@C@C@C@C@C@C@*
0070 0000 0000 0000 0000 0000 0000 0000 0000 *@C@C@C@C@C@C@C@C@C@*
0100 0000 0000 0000 0000 0000 0000 0000 0000 *@C@C@C@C@C@C@C@C@C@*
0110 0000 0000 0000 0000 0000 0000 0000 0000 *@C@C@C@C@C@C@C@C@C@*
0120 0000 0000 0000 0000 0000 0000 0000 0000 *@C@C@C@C@C@C@C@C@C@*
0130 0000 0000 0000 0000 0000 0000 0000 0000 *@C@C@C@C@C@C@C@C@C@*
0140 0000 0000 0000 0000 0000 0000 0000 0000 *@C@C@C@C@C@C@C@C@C@*
0150 0000 0000 0000 0000 0000 0000 0000 0000 *@C@C@C@C@C@C@C@C@C@*
0160 0000 0000 0000 0000 0000 0000 0000 0000 *@C@C@C@C@C@C@C@C@C@*
0170 0000 0000 0000 0000 0000 0000 0000 0000 *@C@C@C@C@C@C@C@C@C@*
```

Figure 4: Dump of Block 0065 of P?S/8 Version 8Z for TC08.

Figure 5 shows the contents of the fourth and final system programs directory block. Relative word[177] contains the first available block for additional system programs. All released P?S/8 systems provide a small number of spare blocks to allow additional programs to be added. There are no utilities available to aid in the creation of additional system programs; advanced users are familiar enough with P?S/8 as documented within this document to successfully add additional programs. In this particular example, blocks 0735-0777 are available for additional system programs.

Note: The TFS directory for this system is the documented example in another document detailing P?S/8 TFS directories. The TFS files for this system start in block 1000.

Temporary system programs can be created by use of the R (or RU{n}) % or R (or RU{n}) \$ command. When binary files are loaded and saved to the system save area (using the GET command or equivalent), it is possible to run the image as if in the System Programs directory via this command, complete with all command-line options; where relevant, the radix for the =xxxx is forced to octal.

The only serious restriction on such usage is the inability to define overlay structures such as are used in P?S/8 PAL and P?S/8 FOCAL; proper implementation requires additional knowledge of P?S/8 internal operations. Additionally, the program must start at location 00200 (which is the most likely address). More information on this form of program execution is available in the *P?S/8 Keyboard Monitor Command Guide* document available separately.

BLOCK: 0:0066										
0000	0000	0000	0000	0000	0000	0000	0000	0000	*@@@@@@@@@@@@@@@@@*	
0010	0000	0000	0000	0000	0000	0000	0000	0000	*@@@@@@@@@@@@@@@@@*	
0020	0000	0000	0000	0000	0000	0000	0000	0000	*@@@@@@@@@@@@@@@@@*	
0030	0000	0000	0000	0000	0000	0000	0000	0000	*@@@@@@@@@@@@@@@@@*	
0040	0000	0000	0000	0000	0000	0000	0000	0000	*@@@@@@@@@@@@@@@@@*	
0050	0000	0000	0000	0000	0000	0000	0000	0000	*@@@@@@@@@@@@@@@@@*	
0060	0000	0000	0000	0000	0000	0000	0000	0000	*@@@@@@@@@@@@@@@@@*	
0070	0000	0000	0000	0000	0000	0000	0000	0000	*@@@@@@@@@@@@@@@@@*	
0100	0000	0000	0000	0000	0000	0000	0000	0000	*@@@@@@@@@@@@@@@@@*	
0110	0000	0000	0000	0000	0000	0000	0000	0000	*@@@@@@@@@@@@@@@@@*	
0120	0000	0000	0000	0000	0000	0000	0000	0000	*@@@@@@@@@@@@@@@@@*	
0130	0000	0000	0000	0000	0000	0000	0000	0000	*@@@@@@@@@@@@@@@@@*	
0140	0000	0000	0000	0000	0000	0000	0000	0000	*@@@@@@@@@@@@@@@@@*	
0150	0000	0000	0000	0000	0000	0000	0000	0000	*@@@@@@@@@@@@@@@@@*	
0160	0000	0000	0000	0000	0000	0000	0000	0000	*@@@@@@@@@@@@@@@@@*	
0170	0000	0000	0000	0000	0000	0000	0000	0735	*@@@@@@@@@@@@@G@]	

Figure 5: Dump of Block 0066 of P?S/8 Version 8Z for TC08.

Tentative definition of the extended-length file entry (for P?S/8 Version 9A).

As a practical matter, the ending words of the fourth system programs directory block are always unused. Since the full definition of the system programs directory is only used by the R (or RU{n}) form of execution, this is ideal placement for the extended-length file entry (as the command execution processor will have already loaded the block into memory).

The tentative definitions are as follows:

Relative words[162-171] are reserved for the scratch file allocation list. This is primarily used by P?S/8 PAL as temporary storage of the cross-reference data if the /X option is in effect. The references are linearly created during the second pass of the assembly; once the main assembly ends, the references are read back in and stored in memory in sorted order for printout after the last linear reference is processed. Note: Multiple passes of the reference data may be necessary on very large assemblies.

Eight words are allocated to allow all possible logical units to provide the extended-length file entries. For any system, the entries consist of the preferred order of the logical units desired. All entries to be allocated will also have the high-order bit on available logical units (expressed in the three low-order bits). During the reference dump process, the actual logical units will be read to determine available output blocks for each unit. Note: Differing amounts will tend to be available depending on system configuration.

Relative words[172-174 are reserved for the extended-length file name. Note: This name must be unique with respect to the TFS directory on the same logical unit; if there is an anomaly, the TFS entry will always take preference.

Relative word[175] is the starting block for the extended-length file.

Relative word[176] is the high limit value for the extended length area on this logical unit.

Note: All extended-length file entry information is optional; on certain systems, extended-length files may not be present.

[end-of-file]